

평G모그의을수o아적치와 방상을방상점을상세부분의점방제성의을식체

Kv 기 결5r 7s 갈5갈빈o 7h경
KdUWK A: A: 5구7?: BB용: CC
Kaot

aot 1a 갈갈갈o 구가검모평케 t 관용 SI l 1 2 rnl a 1r 갈갈갈on 구검구번
l 갈가검갈구판a 갈구갈 7aot 7
rnl a 1구갈갈갈구구검구판케 5aot
7 Uot 7
KX나구검구 aot 5rnl a aot
7 aot 7

SGAN x 과제 o고고

Uot K
Uot
aot K
1cTi 5aWq2
rd W2
K
1 높 2

SGAO x 과제

aot K
레지스터 (Register)
↓ (매우 빠름, 제한적)
공유 메모리 (Shared Memory)
↓ (빠름, 블록 내 공유)
글로벌 메모리 (Global Memory)
↓ (느림, 높은 지연)
메인 메모리 (Host Memory)

K
?7 K
A7 K
B7 K

SGA 알고리즘

1. 공간상의 점들을 K

K

1. Z-order curve (Morton order) 사용
 - 공간을 재귀적으로 분할하며 점들을 순서화
 - 다차원 공간을 1차원으로 인코딩
2. Hilbert curve 사용
 - Z-order보다 공간 지역성 더 잘 보존
 - 더 나은 클러스터링 구조
3. 쿼드트리 기반 정렬
 - 계층적 공간 분할
 - GPU 병렬화에 적합

6. 공간상의 점들을 K

공간상의 점:

- (1, 1) → Z-인덱스 6
- (0, 1) → Z-인덱스 4
- (0, 0) → Z-인덱스 0
- (1, 0) → Z-인덱스 2

Z-순서: (0,0) → (1,0) → (0,1) → (1,1)

SGA

K

```
// 단계 1: 벡터들을 공간 순서대로 정렬
sorted_vectors = spatial_sort(all_vectors)

// 단계 2: 각 벡터에 대해 이웃 연결
for each vector v in sorted_vectors:
    // 로컬 이웃 (공간 순서상 근접)
    local_neighbors = get_spatial_neighbors(v)

    // 글로벌 이웃 (거리 기준)
    global_neighbors = search_nearest(v, K)

    // 하이브리드 이웃 집합
    neighbors[v] = combine(local_neighbors,
                           global_neighbors)

// 단계 3: GPU 최적화를 위한 그래프 리정렬
reorder_for_gpu_cache(graph)
```

SGA 구현

K

```
__global__ void search_batch_kernel(
    const float* queries,      // N_q 개 쿼리
    const float* vectors,     // N 개 벡터
    const int* graph,         // 이웃 관계
    int K, int ef,
    int* results              // 결과 저장
) {
    int query_id = blockIdx.x; // 쿼리별 블록
    int thread_id = threadIdx.x; // 스레드

    // 공유 메모리에 쿼리 벡터 로드
    __shared__ float query[VECTOR_DIM];
```

```

if (thread_id < VECTOR_DIM) {
    query[thread_id] = queries[query_id * VECTOR_DIM
                           + thread_id];
}
__syncthreads();

// 각 스레드가 별도의 벡터와 거리 계산
int vector_id = thread_id;
while (vector_id < N) {
    float dist = compute_distance(
        query,
        vectors[vector_id],
        VECTOR_DIM
    );

    // 결과 업데이트 (원자적 연산)
    atomicUpdate(results, query_id,
                 vector_id, dist);

    vector_id += blockDim.x; // 그리드 스트라이드
}
}

```

SGA

K

```

// 유클리드 거리
distance = sqrt(sum((q_i - v_i)^2))

// GPU에서 병렬 계산
__device__ float compute_l2_distance(
    const float* q, const float* v, int dim
) {
    float sum = 0.0f;

    for (int i = threadIdx.x; i < dim;
         i += blockDim.x) {
        float diff = q[i] - v[i];
        sum += diff * diff;
    }

    // 블록 내 리덕션
    return block_reduce_sum(sum); // sum 병렬 합산
}

```

K

```

similarity = dot(q, v) / (||q|| * ||v||)

// GPU 최적화
// 1. 벡터 정규화 사전 계산
// 2. 내적 계산 병렬화
// 3. 시뮬레이션된 거리 계산 (1 - similarity)

```

SGA

K

배치 크기 = GPU 메모리 / (쿼리당 메모리 + 임시 메모리)

예: GPU 메모리 32GB

쿼리 당 메모리 = 벡터크기 + 결과 + 임시 버퍼
 $\approx 8\text{KB}$ (벡터 차원 1000)

최적 배치 크기 $\approx 4,000,000$ 쿼리

K

시간 →

[배치 1]

H2D 전송 (호스트→GPU)

```

└─ GPU 계산 (검침)
  └─ D2H 전송 (GPU→호스트) [검침]
    [배치 2]
      H2D 전송 [검침]
        └─ GPU 계산
          └─ D2H 전송 [검침]

```

SGAa

K

벡터 데이터: $N * \text{dim} * 4$ 바이트
 그래프 구조: $N * M * 4$ 바이트 (M = 이웃 수)
 정렬 인덱스: $N * 8$ 바이트

 총합: $N * (4 * \text{dim} + 4 * M + 8)$ 바이트

 예: 100만 벡터, 1000차원, $M=16$
 $= 1M * (4K + 64 + 8) = \text{약 } 4GB$

그래프 구조: $N * M * 4$ 바이트 ($M = \text{이웃 수}$)
 정렬 인덱스: $N * 8$ 바이트

총합: $N * (4 * \text{dim} + 4 * M + 8)$ 바이트

예: 100만 벡터, 1000차원, $M=16$
 $= 1M * (4K + 64 + 8) = \text{약 } 4GB$

정렬 인덱스: $N * 8$ 바이트

총합: $N * (4 * \text{dim} + 4 * M + 8)$ 바이트

예: 100만 벡터, 1000차원, $M=16$
 $= 1M * (4K + 64 + 8) = \text{약 } 4GB$

총합: $N * (4 * \text{dim} + 4 * M + 8)$ 바이트

예: 100만 벡터, 1000차원, M=16

$= 1M * (4K + 64 + 8) = \text{약 } 4GB$

예: 100만 벡터, 1000차원, M=16
= 1M * (4K + 64 + 8) = 약 4GB

$$= 1\text{M} * (4\text{K} + 64 + 8) = \text{약 } 4\text{GB}$$

aot K

aot 1S?::2KC: 1H1: aT

aot 1qsx2KH1:ACaT

aot 1asx2KA1:CaT

aot 1S?: : 2KC: 3H: aT
aot 1qsx 2KH 3HACaT
aot 1bsx 2KA 3HFaT

aot 1sx2KH₂CaT

aot 1sx2KA₂CaT

aot 1asx2KA₅EaT

SGA α r 곱하기 x 곱하기[illegible]

K

$$K? : \frac{L?}{H?} ::$$

K

SGANG

```

aot                                     K
__global__ void search_filtered_kernel(
    const float* queries,
    const float* vectors,
    const int* attributes,      // 속성값
    const AttributeFilter* filters,
    int* results
) {
    int query_id = blockIdx.x;
    int vector_id = threadIdx.x;

    // 1. 필터 조건 확인 (GPU에서 빠름)
    if (!check_filter(attributes[vector_id], filters)) {
        return; // 필터링된 벡터 스킵
    }

    // 2. 거리 계산
    float dist = compute_distance(
        queries[query_id],
        vectors[vector_id]
    );

    // 3. 결과 저장

```

```
__global__ void search_filtered_kernel(
    const float* queries,
    const float* vectors,
    const int* attributes,    // 속성값
    const AttributeFilter* filters,
    int* results
) {
    int query_id = blockIdx.x;
    int vector_id = threadIdx.x;

    // 1. 필터 조건 확인 (GPU에서 빠름)
    if (!check_filter(attributes[vector_id], filters)) {
        return; // 필터링된 벡터 스킵
    }

    // 2. 거리 계산
    float dist = compute_distance(
        queries[query_id],
        vectors[vector_id]
    );

    // 3. 결과 저장
```

```
) {
    int query_id = blockIdx.x;
    int vector_id = threadIdx.x;

    // 1. 필터 조건 확인 (GPU에서 빠름)
    if (!check_filter(attributes[vector_id], filters)) {
        return; // 필터링된 벡터 스킵
    }

    // 2. 거리 계산
    float dist = compute_distance(
        queries[query_id],
        vectors[vector_id]
    );

    // 3. 결과 저장
```

```
int query_id = blockIdx.x;
int vector_id = threadIdx.x;

// 1. 필터 조건 확인 (GPU에서 빠름)
if (!check_filter(attributes[vector_id], filters)) {
    return; // 필터링된 벡터 스킵
}

// 2. 거리 계산
float dist = compute_distance(
    queries[query_id],
    vectors[vector_id]
);

// 3. 결과 저장
```

```
int vector_id = threadIdx.x;

// 1. 필터 조건 확인 (GPU에서 빠름)
if (!check_filter(attributes[vector_id], filters)) {
    return; // 필터링된 벡터 스킵
}

// 2. 거리 계산
float dist = compute_distance(
    queries[query_id],
    vectors[vector_id]
);

// 3. 결과 저장
```

```
// 1. 필터 조건 확인 (GPU에서 빠름)
if (!check_filter(attributes[vector_id], filters)) {
    return; // 필터링된 벡터 스킵
}

// 2. 거리 계산
float dist = compute_distance(
    queries[query_id],
    vectors[vector_id]
);

// 3. 결과 저장
```

```

    if (!check_filter(attributes[vector_id], filters)) {
        return; // 필터링된 벡터 스킵
    }

    // 2. 거리 계산
    float dist = compute_distance(
        queries[query_id],
        vectors[vector_id]
    );

    // 3. 결과 저장

```

```
}

// 2. 거리 계산
float dist = compute_distance(
    queries[query_id],
    vectors[vector_id]
);

// 3. 결과 저장
```

```
// 2. 거리 계산
float dist = compute_distance(
    queries[query_id],
    vectors[vector_id]
);

// 3. 결과 저장
```

```
float dist = compute_distance(
    queries[query_id],
    vectors[vector_id]
);

// 3. 결과 저장
```

```
// 3. 결과 저장
```

// 3. 결과 저장

```
atomicUpdate(results, query_id, vector_id, dist);
}
```

SGANN 프로그

K

K

1 2
1 2
1 2

K

aot K? 8DsT8근까Uot ?::aT8근2

SGANN 프로그

1가문개K

aot

KUot

K

aot

K

8

7	aot	K	aot	P
P				
A7		K	5	P
B7aot		K		P
C7	aot	K	aot	P
D7		Kaot		P
E7		K	1 2aot	P
G7	K		aot	P
H7		KUot	aot	P
